

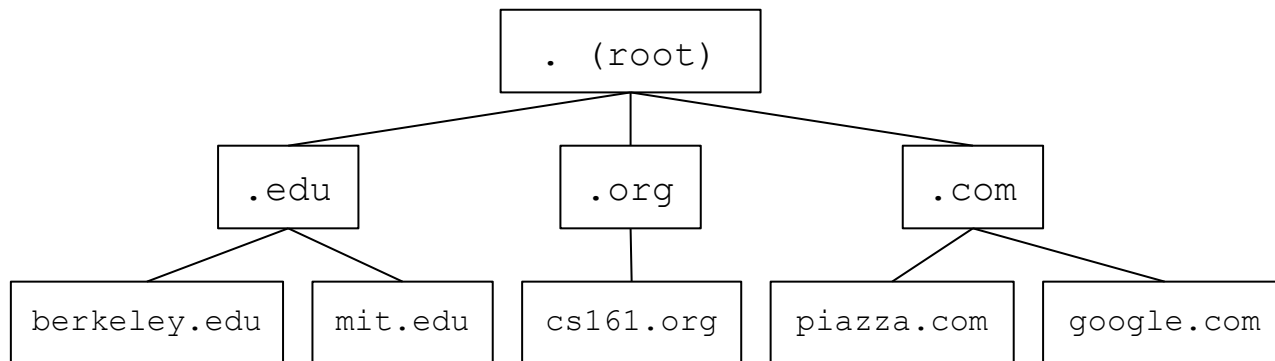
# DNSSEC

CSE 405 January 2025 - Lecture 12

Special Thanks to Peyrin Kao (UC Berkeley)

# Last Time: DNS Overview

- DNS (Domain Name System): An Internet protocol for translating human-readable domain names to IP addresses
  - DNS name servers on the Internet provide answers to DNS queries
  - Name servers are arranged in a domain hierarchy tree
  - Lookups proceed down the domain tree: name servers will direct you down the tree until you receive an answer
  - The stub resolver tells the recursive resolver to perform the lookup



# Last Time: DNS Implementation

- DNS message structure
  - DNS uses UDP for efficiency
  - DNS packets include a random 16-bit ID field to match requests to responses
  - Data is encoded in records, which are name-value pairs with a type
    - **A (answer) type records**: Maps a domain name to an IPv4 address
    - **NS (name server) type records**: Designates another DNS server to handle a domain
  - Records are separated into four sections
    - Question: Contains query
    - Answer: Contains direct answer to query
    - Authority: Directs the resolver to the next name server
    - Additional: Provides extra information (e.g. the location of the next name server)
  - Resolvers cache as many records as possible (until their time-to-live expires)

# Last Time: DNS Security

- Cache poisoning attack: Send a malicious record to the resolver, which caches the record
  - Causes packets to be sent to the wrong place (e.g., to the attacker, who becomes a MITM)
- Risk: Malicious name servers
  - Defense: Bailiwick checking: Resolver only accepts records in the name server's zone
- Risk: Network attackers
  - MITM attackers can poison the cache without detection
  - On-path attackers can race the legitimate response to poison the cache
  - Off-path attackers must guess the ID field (Defense: Make the ID field random)
    - Kaminsky attack: Query non-existent domains and put the poisoned record in the additional section (which will still be cached). Lets the off-path attacker try repeatedly until succeeding
    - Defense: Source port randomization (more bits for the off-path attacker to guess)

# Outline

- DNS over TLS
  - Issues
- DNSSEC
  - High-level design
  - Design details
  - Implementation details
  - Key-signing keys and zone-signing keys
  - NSEC: Signing non-existent domains
  - In practice



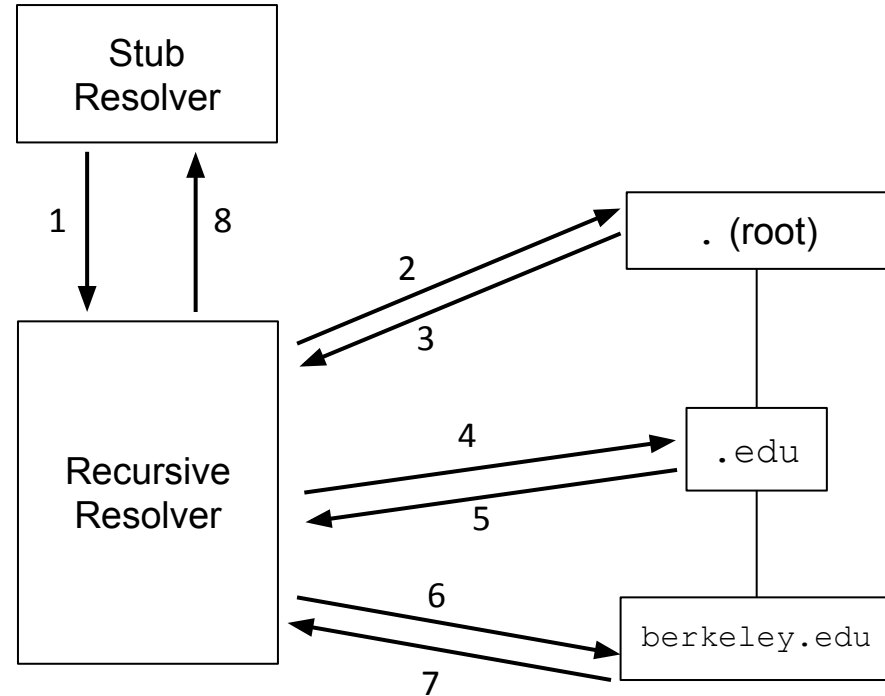
# DNS over TLS

# Securing DNS Lookups

- Recall: DNS is not secure against several threats
  - Malicious name servers
  - Network attackers (MITM, on-path, off-path)
- We want **integrity** on the response
  - Recall: Integrity means an attacker can't tamper with the results
  - Prevents cache poisoning attacks
- We do not need **confidentiality** on the response
  - DNS results are public: The attacker can always look up the results themselves!
  - Even if the attacker couldn't see the DNS response, they can still see which IP you connect to later

# DNS over TLS

- Idea: TLS is end-to-end secure, so let's send all DNS requests and responses over TLS



# DNS over TLS: Issues

- **Performance:** DNS needs to be lightweight and fast. TLS is slow.
  - Recall: TLS requires a long cryptographic handshake before any messages can be sent
- **Caching:** DNS records are cached. TLS doesn't help us with caching.
  - What if someone changes the record while it's stored in the cache?
- **Security:** DNS over TLS doesn't defend against malicious name servers.
  - A malicious name server can still poison the cache
- **Security:** DNS over TLS doesn't defend against malicious recursive resolvers.
  - The recursive resolver is a full MITM: a malicious recursive resolver can poison the cache before returning the result to the user
  - The recursive resolver is the most common MITM adversary in DNS

# Object Security and Channel Security

- Main problem: DNS over TLS secures the communication channel, but doesn't help you trust who you're talking to
  - Example: TLS secures your communication with the recursive resolver, but you still need to implicitly trust the recursive resolver. What if the recursive resolver is malicious?
- **Channel security**: Securing the communication channel between two end hosts
- **Object security**: Securing a piece of data (in transit or in storage)
- TLS provides channel security, but to secure DNS, we need object security

# DNS over TLS in Practice

- Recently introduced by Firefox
  - Enabled by default in the United States
- Benefits
  - The added security is worth the slower performance
  - The performance impact is less noticeable now that network speeds are faster
- Drawbacks
  - Only defends against network attackers, not malicious name servers
  - Network attackers can perform a **downgrade attack**: Block the TLS connection, forcing the browser to fall back on ordinary DNS
- DNS over TLS traffic is routed through Cloudflare
  - Cloudflare is a full MITM
  - The only protection is contractual: Cloudflare promises not to misuse your data
- **Takeaway**: DNS over TLS is not enough to fully secure DNS

# DNSSEC: High-Level Design

# DNSSEC

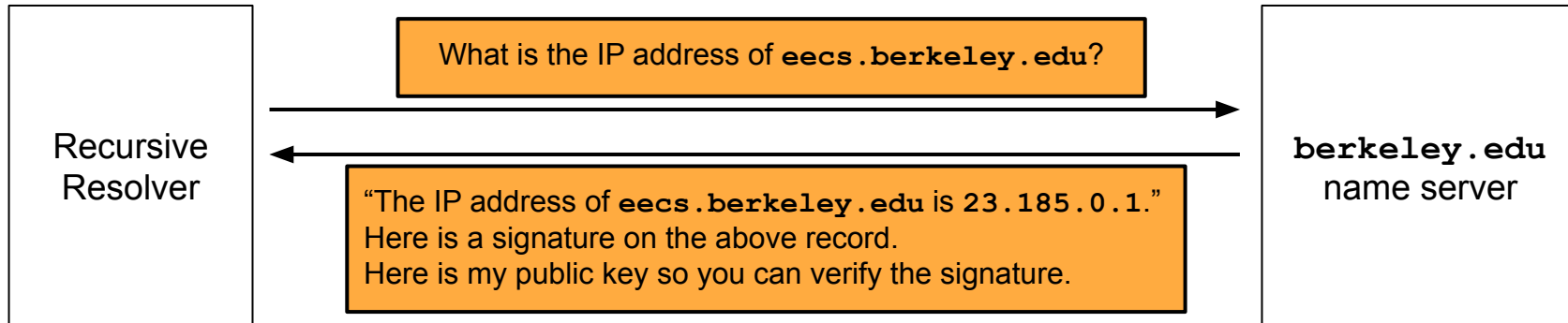
- **DNSSEC (DNS Security Extensions):** An extension of the DNS protocol that ensures integrity on the results
  - Designed to cryptographically prove that returned answers are correct
  - Uses a hierarchical, distributed trust system to validate records
- **DNSSEC is backwards-compatible**
  - Some, but not all name servers support DNSSEC
  - DNSSEC is built on top of ordinary DNS

# Scratchpad: Let's Design It Together

- Question 1: What kind of cryptographic primitive should we use to ensure integrity on the records?
  - We should use a scheme that provides integrity: either MACs (symmetric-key) or digital signatures (public-key)
  - Digital signatures are the best solution here: We want everyone to be able to verify integrity (not just the people with the symmetric key)
- Question 2: How do we ensure the returned record is correct and has not been tampered?
  - Digital signatures: The authoritative name server should sign the record with their private key. Everyone can verify the record with their public key.

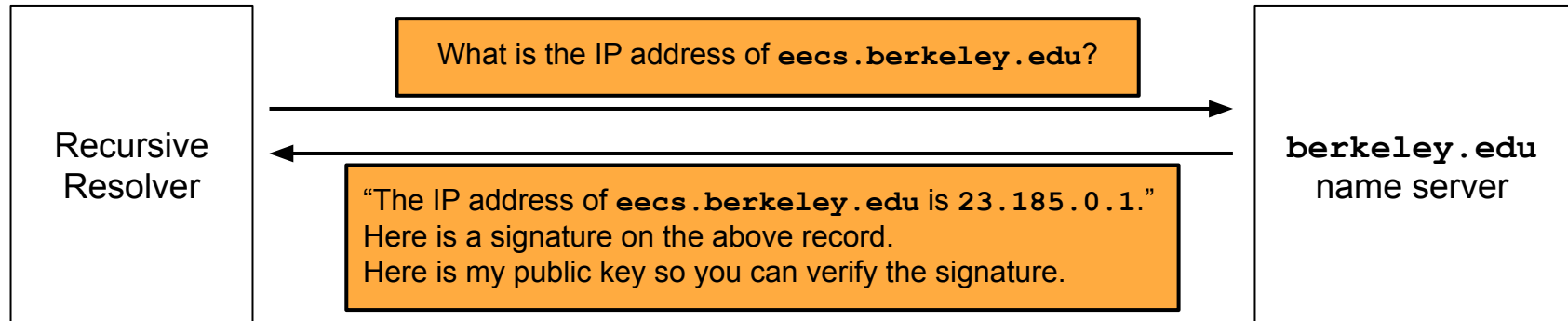
# Scratchpad: Let's Design It Together

- Question 3: What does the name server need to send in order to ensure integrity on a record?
  - The record
  - A signature over the record, signed with the private key
  - The public key



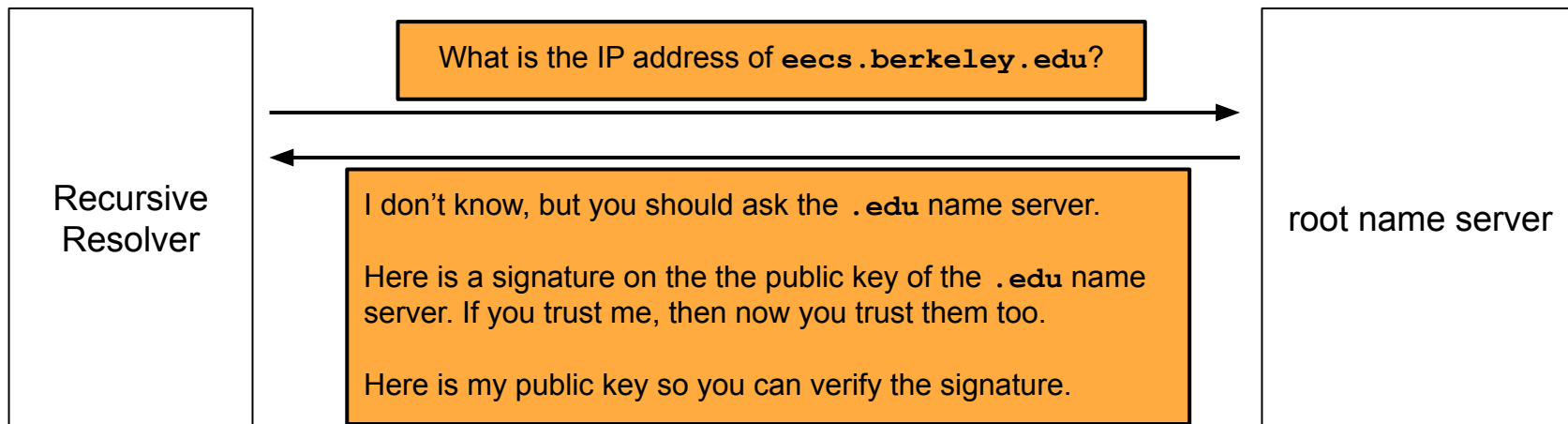
# Issues With This Design

- What if the name server is malicious? They could still return malicious records and sign them.
- How do we make sure nobody tampered with the public key?
- Do these sound like problems that we've solved before in this class?



# Scratchpad: Let's Design It Together

- Question 4: How does a name server delegate trust to a child name server?
  - Just like in a certificate chain, the parent must sign the child's public key.
- Question 5: PKIs need a trust anchor. Who do we implicitly trust in DNSSEC?
  - We implicitly trust the top of the certificate hierarchy, which is the root name server.



# DNSSEC: Design Details

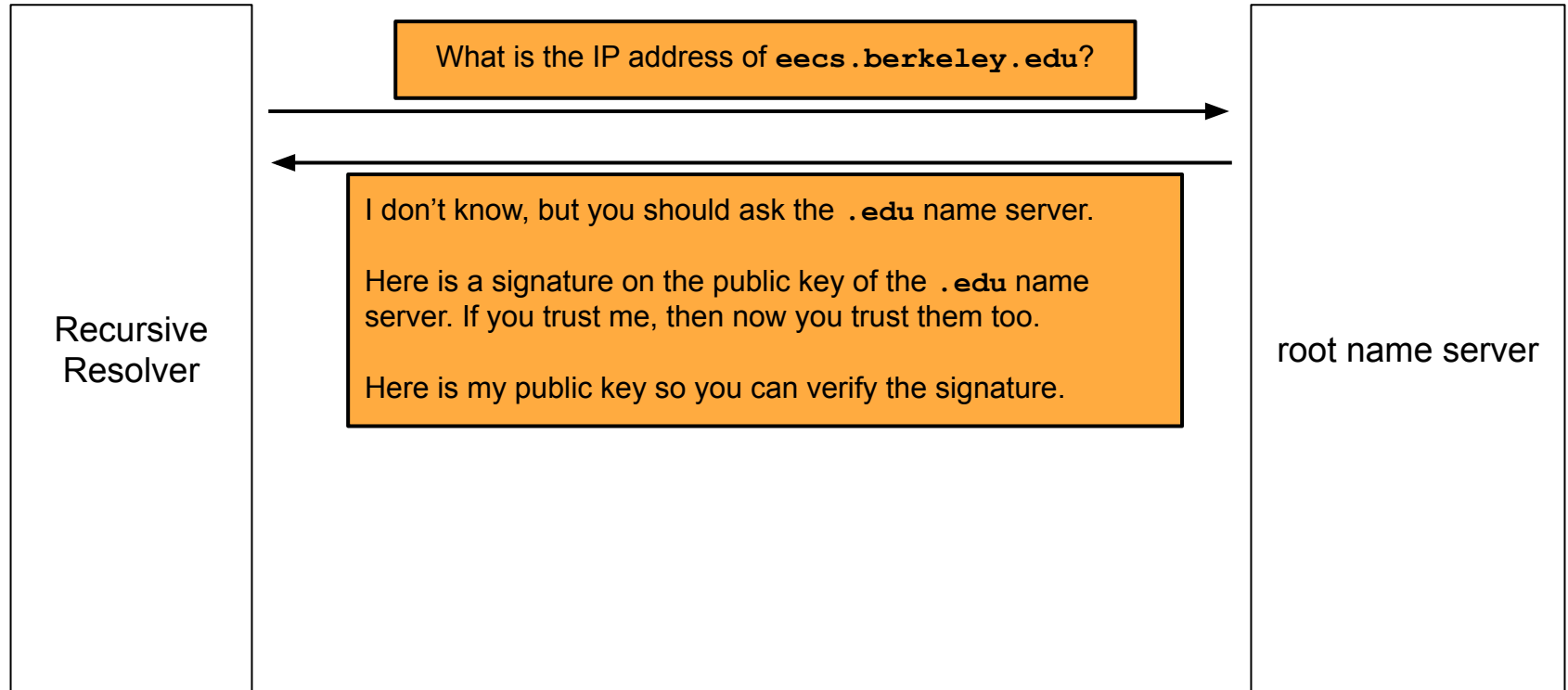
# Idea #1: Sign Records

- Digital signatures provide integrity
  - Only the name server with the private key can generate signatures
  - Everybody can verify signatures with the public key
- Digital signatures defeat network attackers
  - An off-path, on-path, or MITM attacker can no longer tamper with records
  - The recursive resolver can no longer tamper with records
- Signatures can be cached with the records for object security
  - Any time we fetch a record from the cache, we can verify its integrity

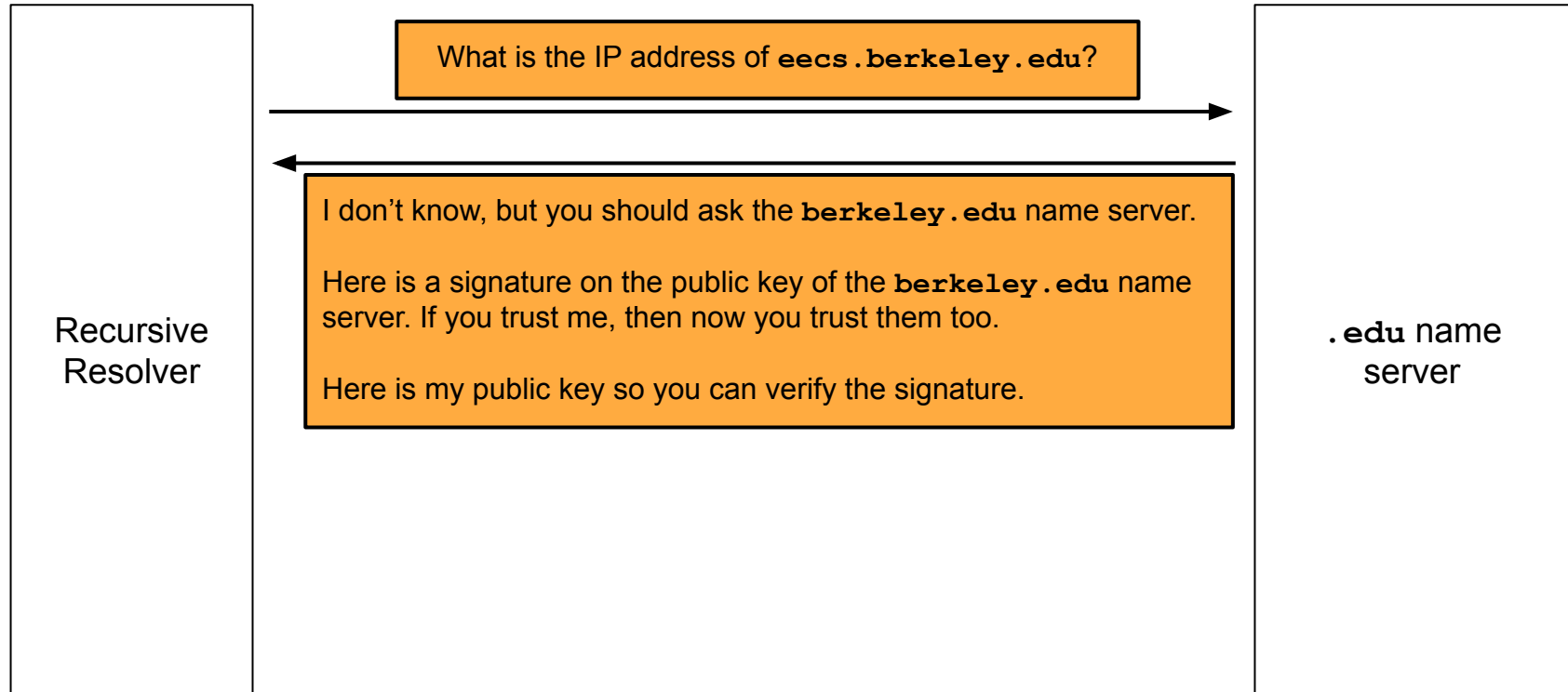
# Idea #2: Public-Key Infrastructure (PKI)

- Name servers are arranged in a hierarchy, as in ordinary DNS
- Parents can delegate trust to children
  - The parent signs the child's public key to delegate trust to the child
  - If you trust the parent name server, then now you trust the child name server
- Trust anchor: We implicitly trust the root name server
  - The root name server's public key is hard-coded into resolvers
- PKI defeats malicious name servers
  - A malicious name server (assuming they don't have access to the private key, only the signatures) won't have a valid chain of trust back to the root

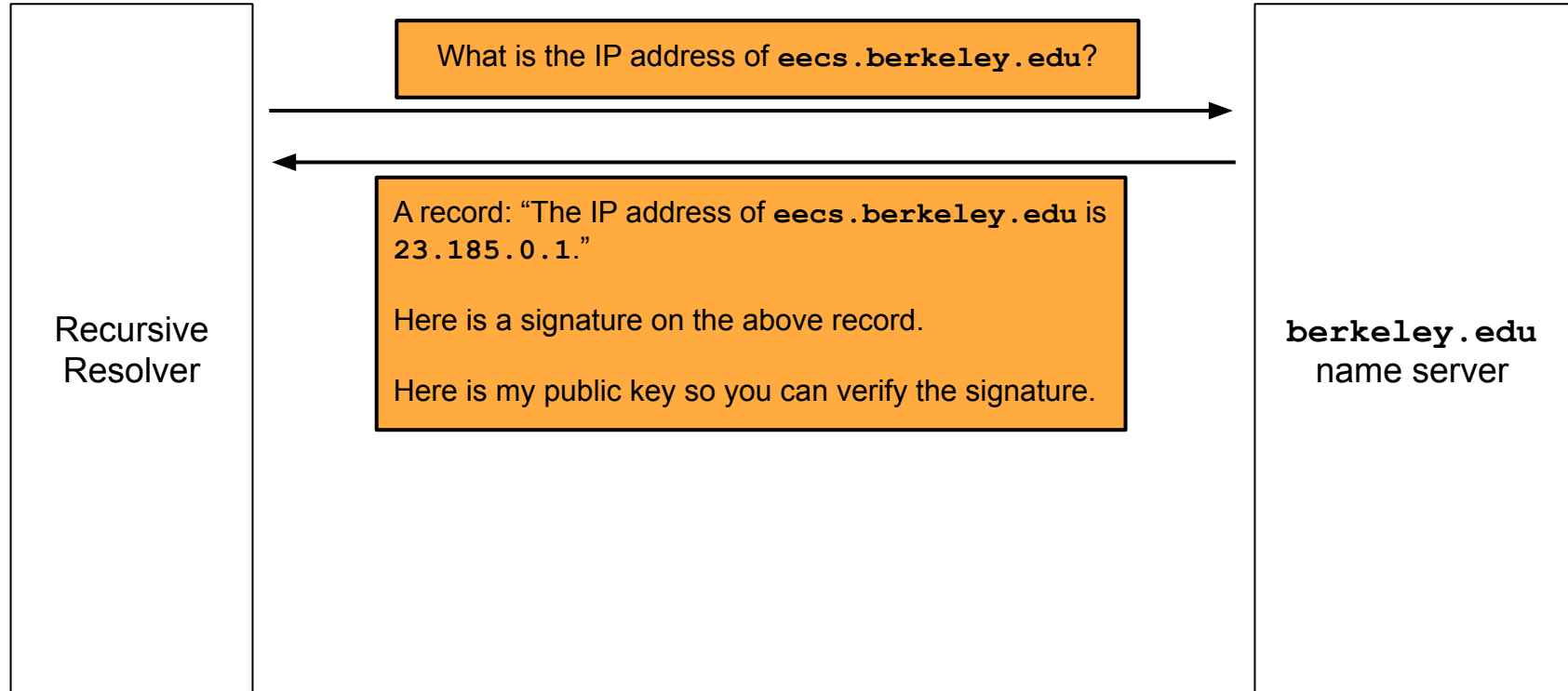
# Steps of a DNSSEC Lookup (Attempt #1)



# Steps of a DNSSEC Lookup (Attempt #1)



# Steps of a DNSSEC Lookup (Attempt #1)



# DNSSEC: Implementation

# Warning: Unfiltered DNSSEC Ahead

CSE 405 - Computer Security - ART

- We're now going to show you the entire DNSSEC protocol, with all its implementation details and edge cases.
- Some parts are less important for the intuition of DNSSEC and won't be tested on exams. We're going to highlight these parts in blue.

# Review: DNS Packet Format

- The DNS header contains metadata about the query (e.g. ID number, flags)
- There are 8 bits for flags

|                    |                  |             |
|--------------------|------------------|-------------|
| Source Port        | Destination Port | UDP Header  |
| Checksum           | Length           |             |
| ID number          | Flags            | DNS Header  |
| Question count     | Answer count     |             |
| Authority count    | Additional count |             |
| Question Records   |                  | DNS Payload |
| Answer Records     |                  |             |
| Authority Records  |                  |             |
| Additional Records |                  |             |

# OPT Pseudosection

- Ordinary DNS has size limits
  - 8 bits for flags
  - Messages are limited to 512 bytes
- DNSSEC messages exceed these limits
  - Additional flags needed in DNSSEC
    - **DO** flag indicates we support DNSSEC and want DNSSEC records
    - **CD** flag indicates we support DNSSEC, but we don't want to verify the DNSSEC signatures for now
  - Messages are larger than 512 bytes
- Remember: We want DNSSEC to be backwards-compatible
  - We can't modify the existing DNS limits! What should we do?

# OPT Pseudosection

- Solution: Encode extra flags in a record called the **OPT Pseudosection**
  - This record has type OPT
  - This record is sent in the additional section
- **EDNS0 (Extension Mechanisms for DNS)**: The protocol that adds the OPT pseudosection
  - If DNSSEC is enabled, the resolver sends the OPT record in the request, and the name server sends the OPT record in the reply
  - The OPT pseudosection can be used to specify the size of larger UDP replies
- **Takeaway**: We found a way to add extra functionality to DNSSEC while supporting ordinary DNS (backwards compatibility)

# Resource Record Sets (RRSETs)

- Recall: A DNS record has a name, type, and value
- A group of DNS records with the same name and type form a **resource record set (RRSET)**
  - Example: All the AAAA records for a given domain
- RRSETs will be useful for simplifying signatures
  - Instead of signing every record separately, we can sign an entire RRSET at once

# New DNSSEC Record Types

- We need new record types to send cryptographic information in DNSSEC packets
  - RRSIG (resource record signature): encode signatures on records
  - DNSKEY: encode public keys
  - DS (delegated signer): encode the child's public key (used to delegate trust)

# New DNSSEC Record Types: RRSIG

- RRSIG type records encode a signature on records
  - One RRSIG record (with one signature) can sign an entire RRSET
- RRSIG type records contain some additional metadata
  - Type: What type of DNS record we're signing
  - Algorithm: What algorithm we're using to create the signature
  - Label: Number of segments in the DNS name
  - Original TTL: The TTL for the records in the RRSET
  - Signature expiration time (in Unix time: seconds since January 1, 1970)
  - Signature inception time: When the signature was created (in Unix time)
  - Key tag: What key was used (roughly, a checksum on key bits)
  - The name of the signer

# New DNSSEC Record Types: DNSKEY

- DNSKEY type records encode the name server's own public keys
- DNSKEY type records contain some additional metadata too
  - 16 bits of flags
  - Protocol identifier (currently not in use, so always set to 3)
  - Algorithm identifier

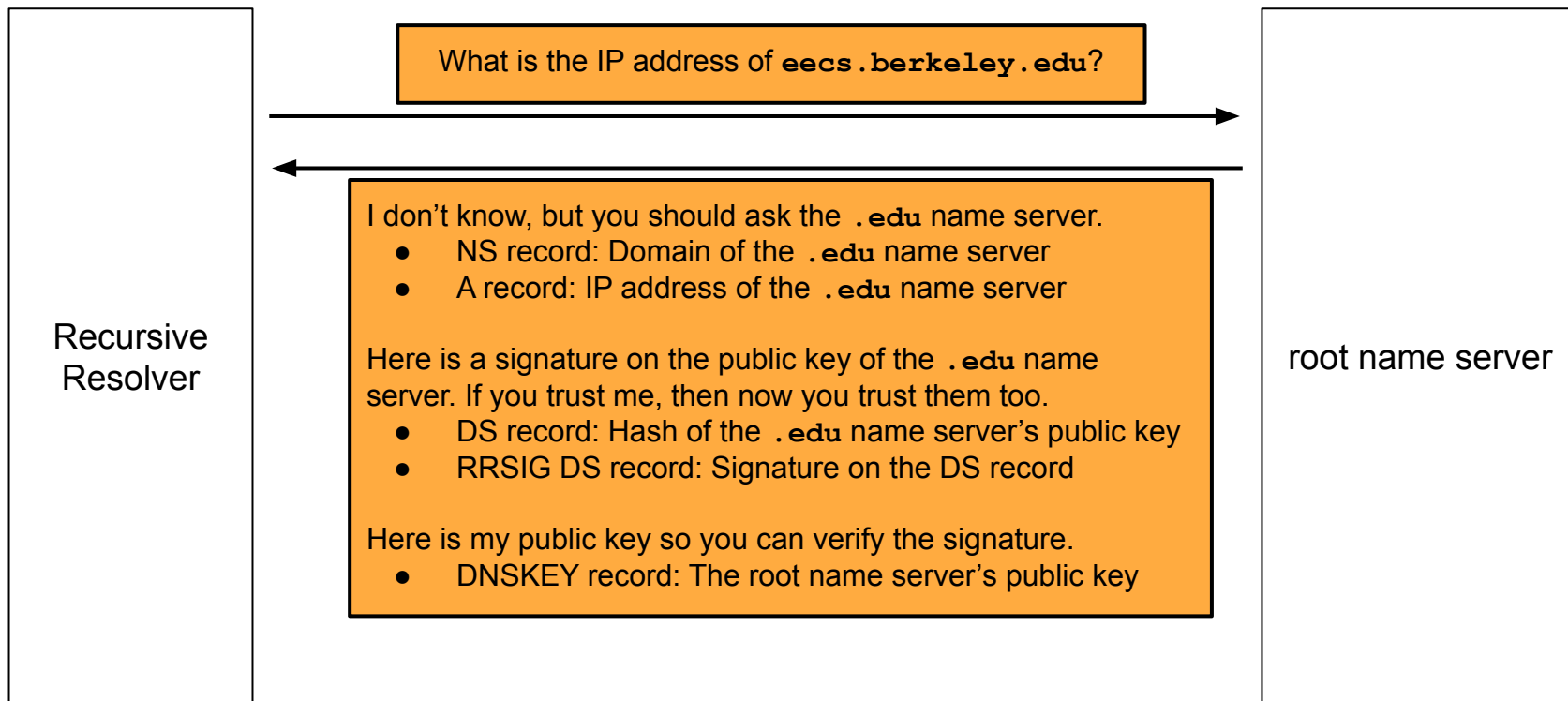
# New DNSSEC Record Types: DS

- DS type records encode the hash of the child's public keys
  - Used to delegate trust
- DS type records contain some additional metadata too
  - The key tag
  - The algorithm identifier
  - The hash function used (we'll see this next)
- **Takeaway:** Real-world protocols like DNSSEC require a lot of metadata to function correctly!
  - It's usually pretty uninteresting, though, which is why we abstract it away for you

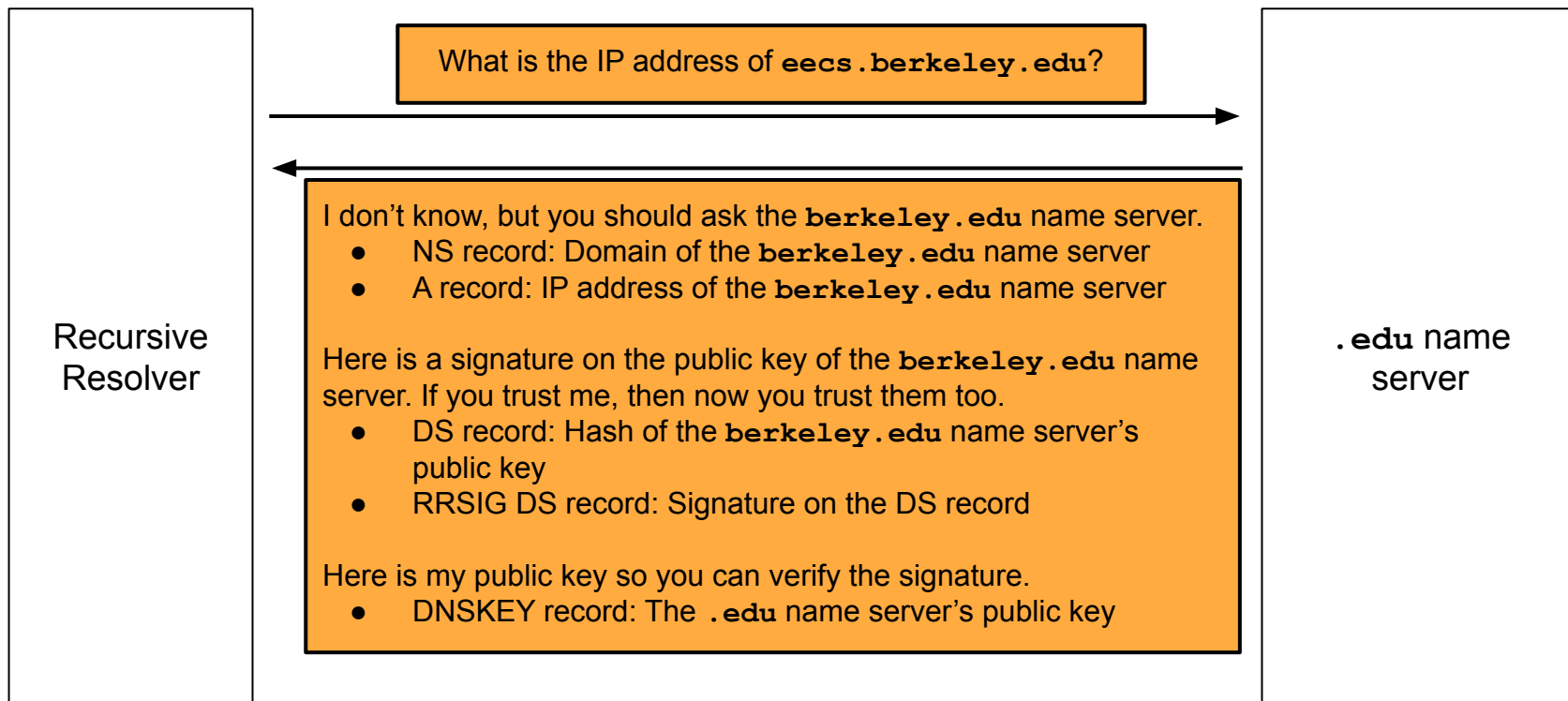
# New DNSSEC Record Types: Trust Delegation

- Recall delegating trust: The parent signs the child's public key to delegate trust to the child
- DNSSEC delegates trust with two records:
  - A DS type record with the hash of the signer's name and the child's public key
  - An RRSIG type record with a signature on the DS record

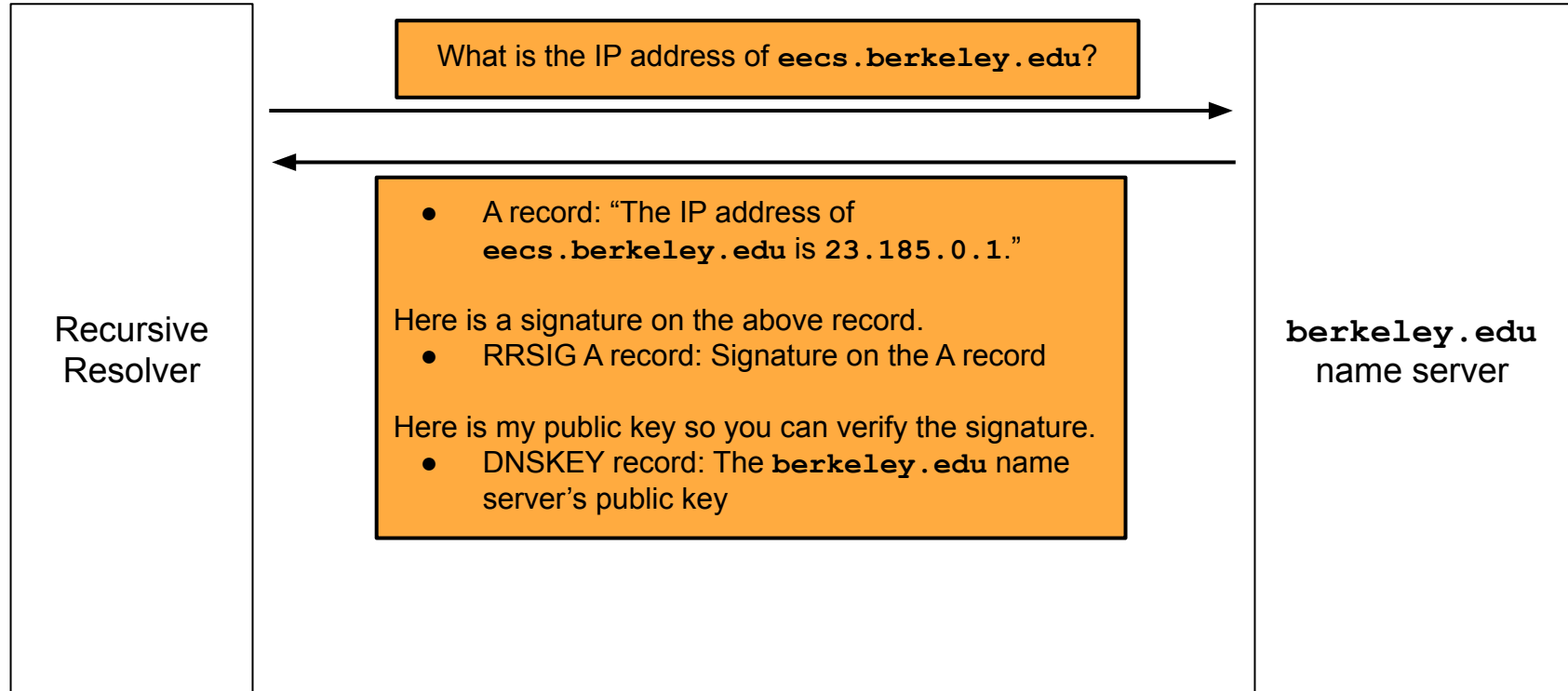
# Steps of a DNSSEC Lookup (Attempt #2)



# Steps of a DNSSEC Lookup (Attempt #2)



# Steps of a DNSSEC Lookup (Attempt #2)



# Key-Signing Keys and Zone-Signing Keys

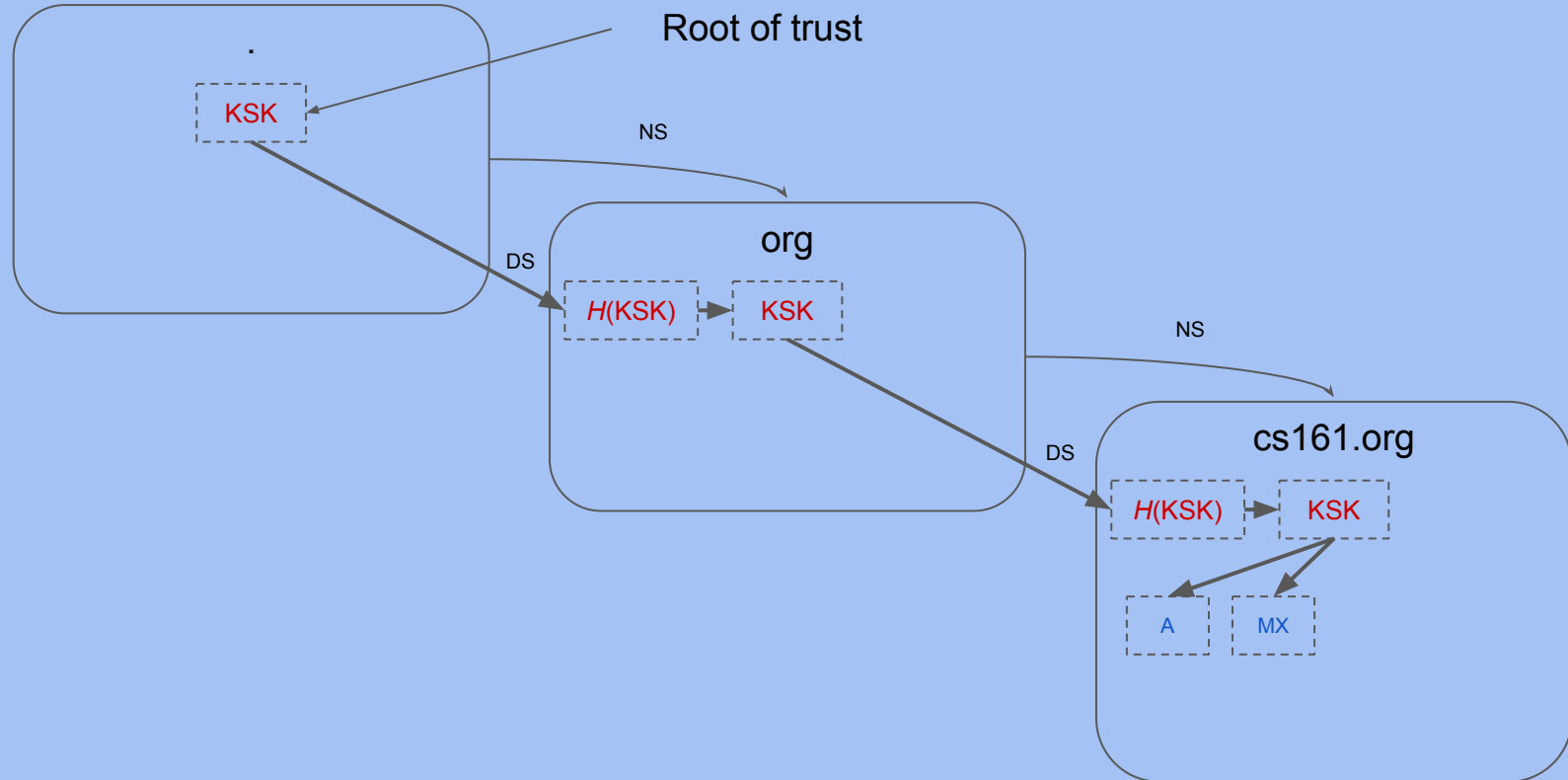
# Motivation: Recovering from Key Compromise

- What if a name server wants to change the keys it uses to sign records?
  - Example: This is necessary if the attacker compromises a private key
- The name server needs to inform its parent, since the parent must change its DS record too!
  - This process is complicated and can go wrong in many ways
  - We want to avoid this process whenever possible
- Solution: Divide each name server into an *upper half* and *lower half*
  - If we need to change the keys in the lower half, we don't need to contact another name server: the parent is the upper half of the *same* name server!

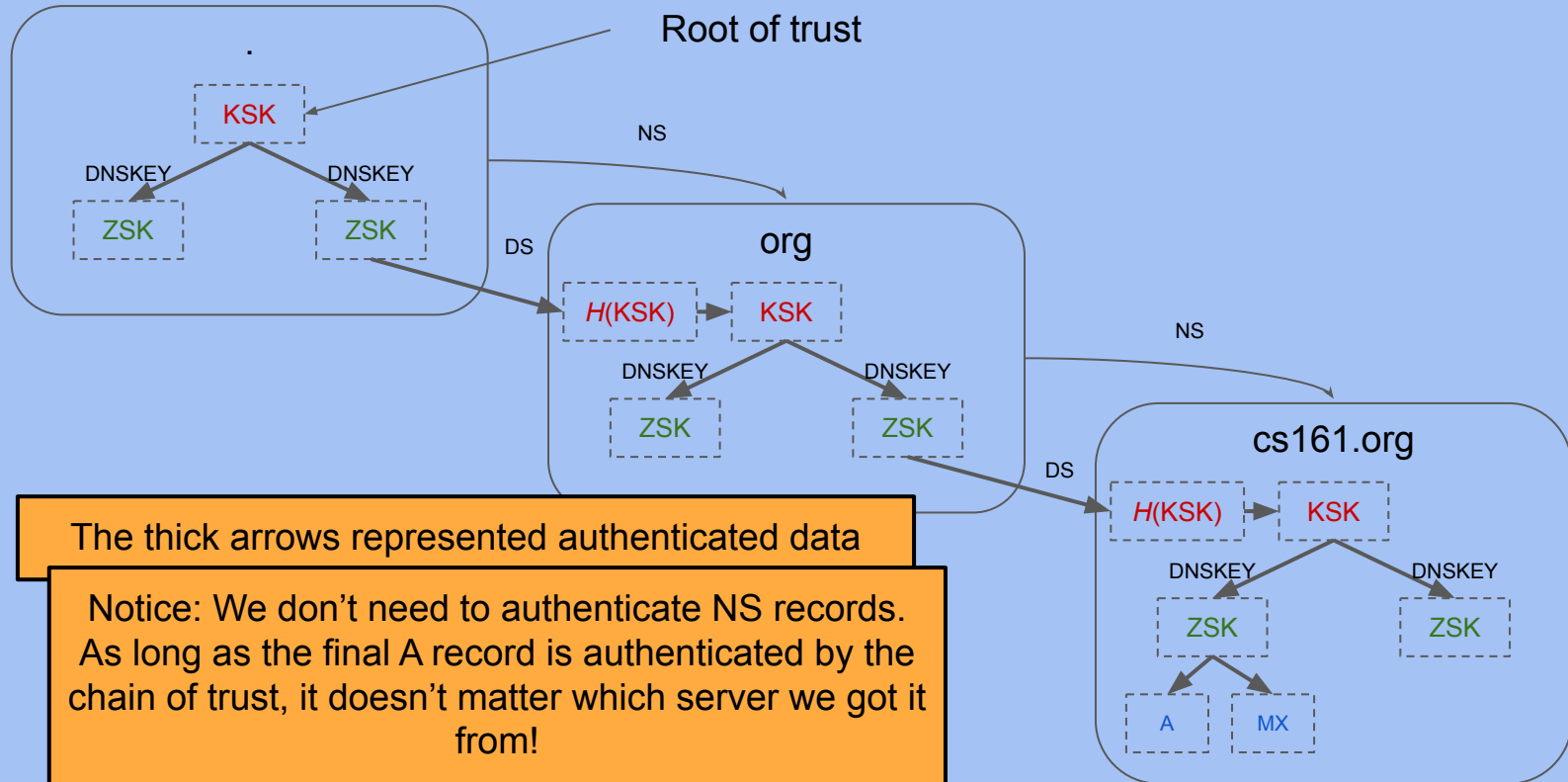
# Key-Signing Keys and Zone-Signing Keys

- Each name server has *two* kinds of public-private key pairs
- The **key-signing key (KSK)** is used to sign only the zone-signing key
  - Intuition: The KSK is the “upper half” of the name server.
  - The “upper half” endorses the “lower half”
- The **zone-signing key (ZSK)** is used to sign all other records
  - Intuition: The ZSK is the “lower half” of the name server
  - The “lower half” endorses the “upper half” of the next name server (or the final answer)
- Example
  - Now, the **berkeley.edu** name server has two key pairs (KSK and ZSK)
  - The private KSK is used to sign the public ZSK
  - The private ZSK is used to sign the final A record

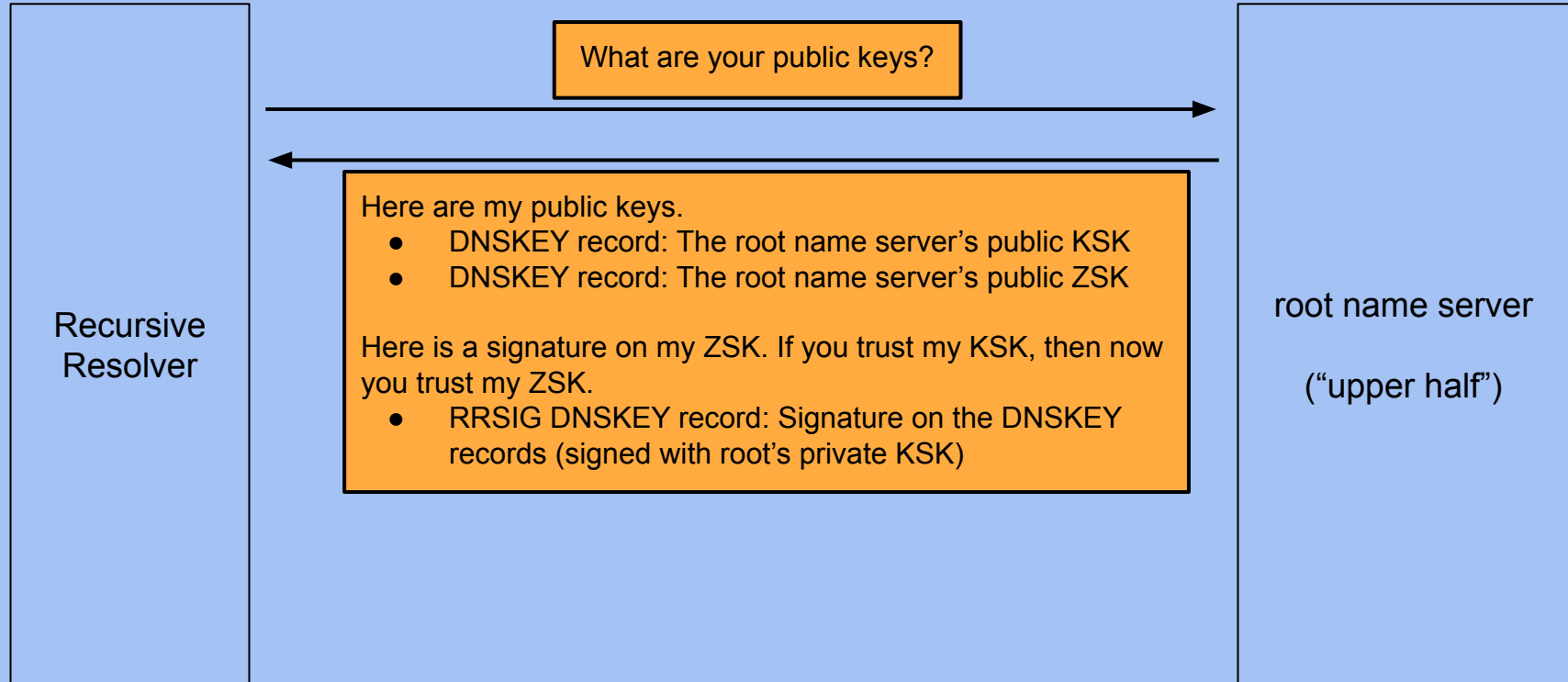
# Path of Trust (without KSKs and ZSKs)



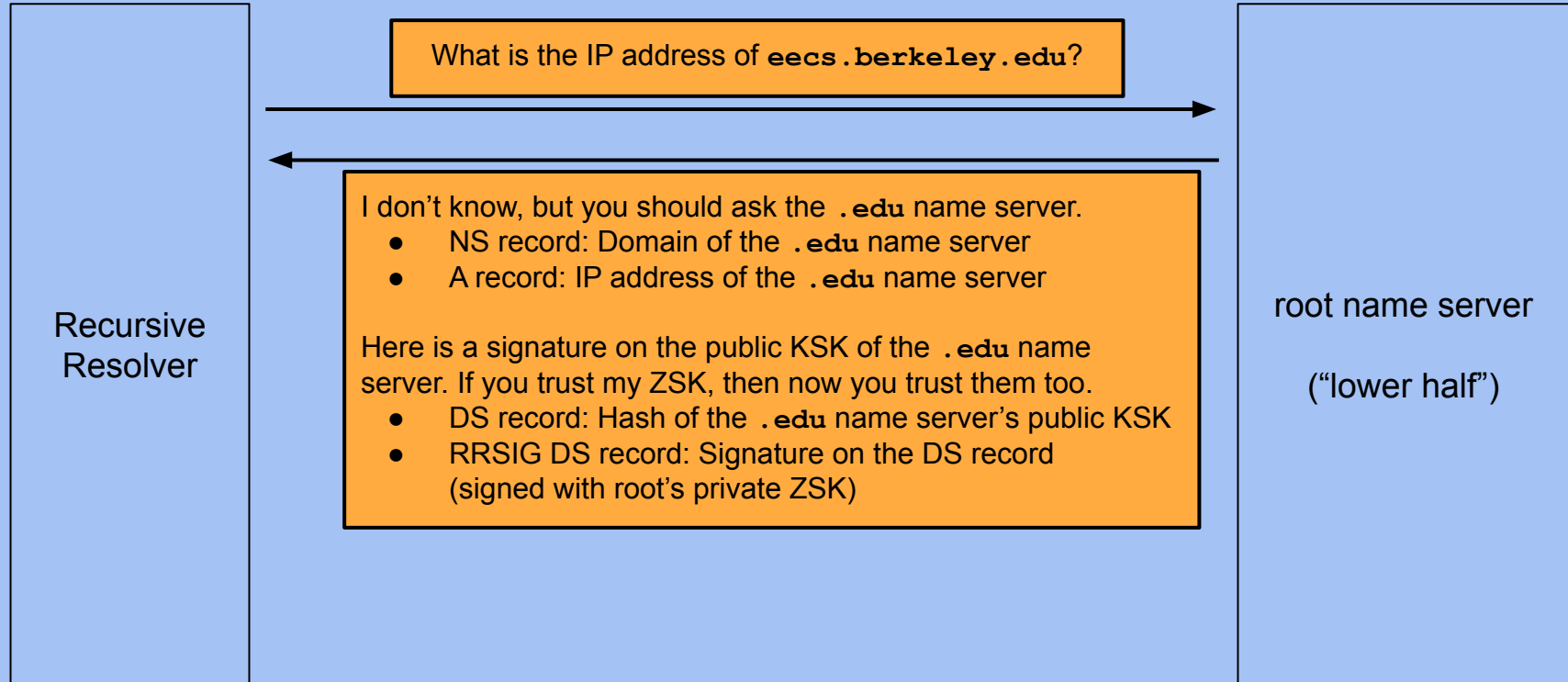
# Path of Trust (with KSKs and ZSKs)



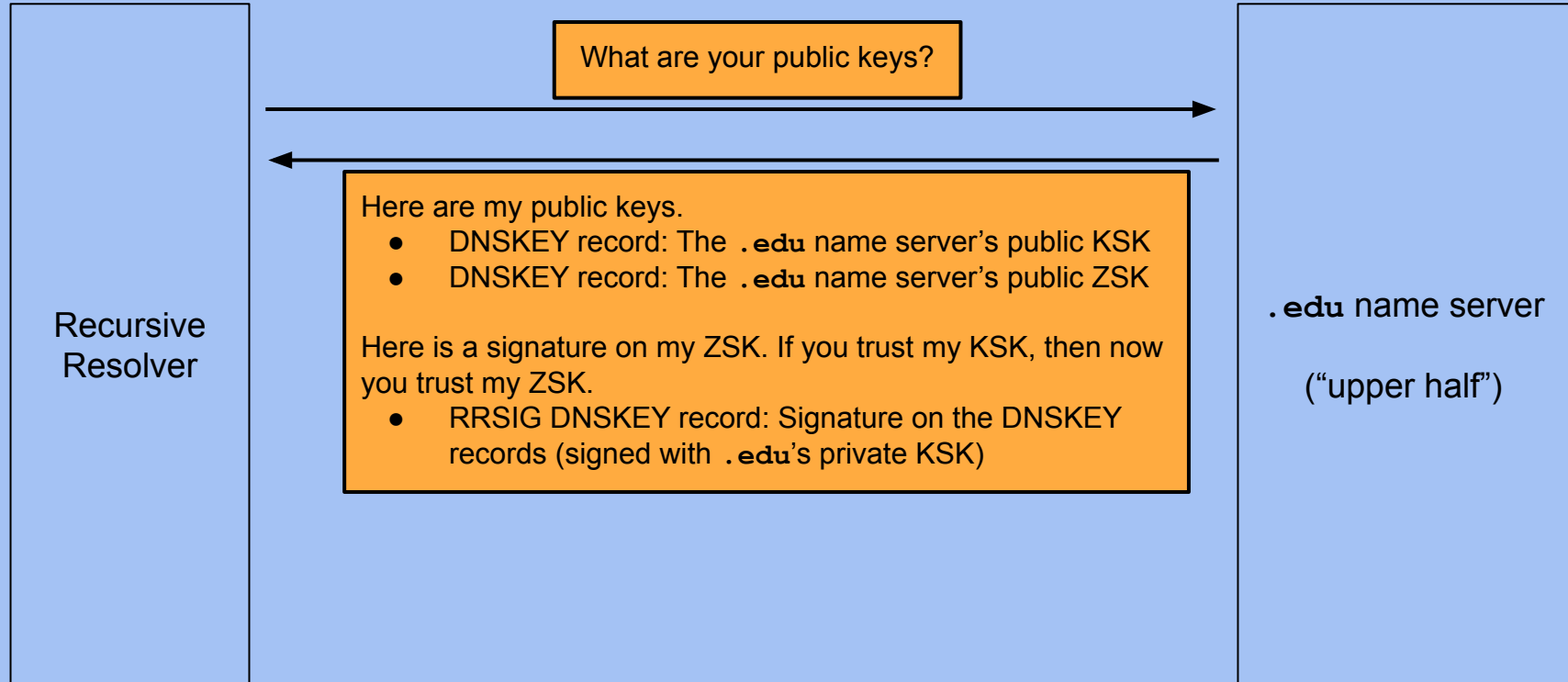
# Steps of a DNSSEC Lookup (Attempt #3)



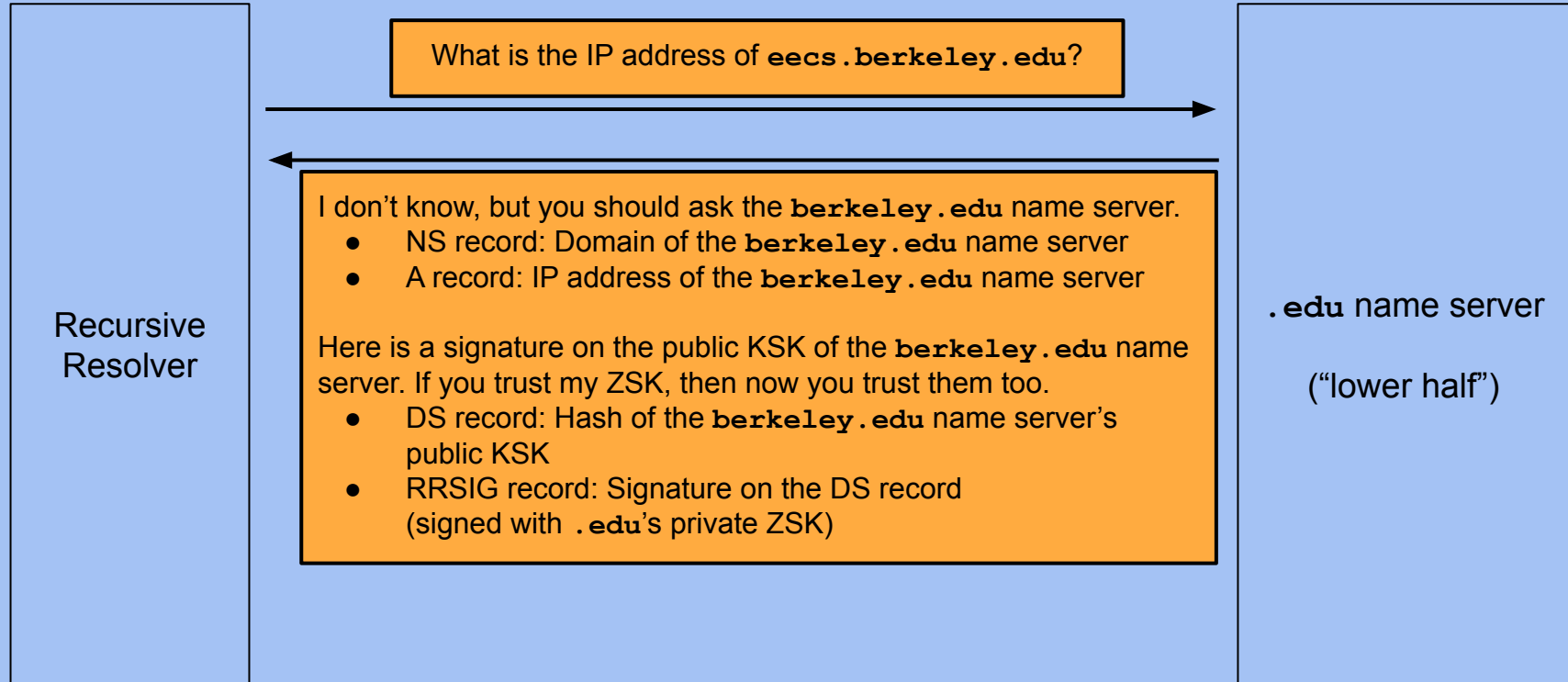
# Steps of a DNSSEC Lookup (Attempt #3)



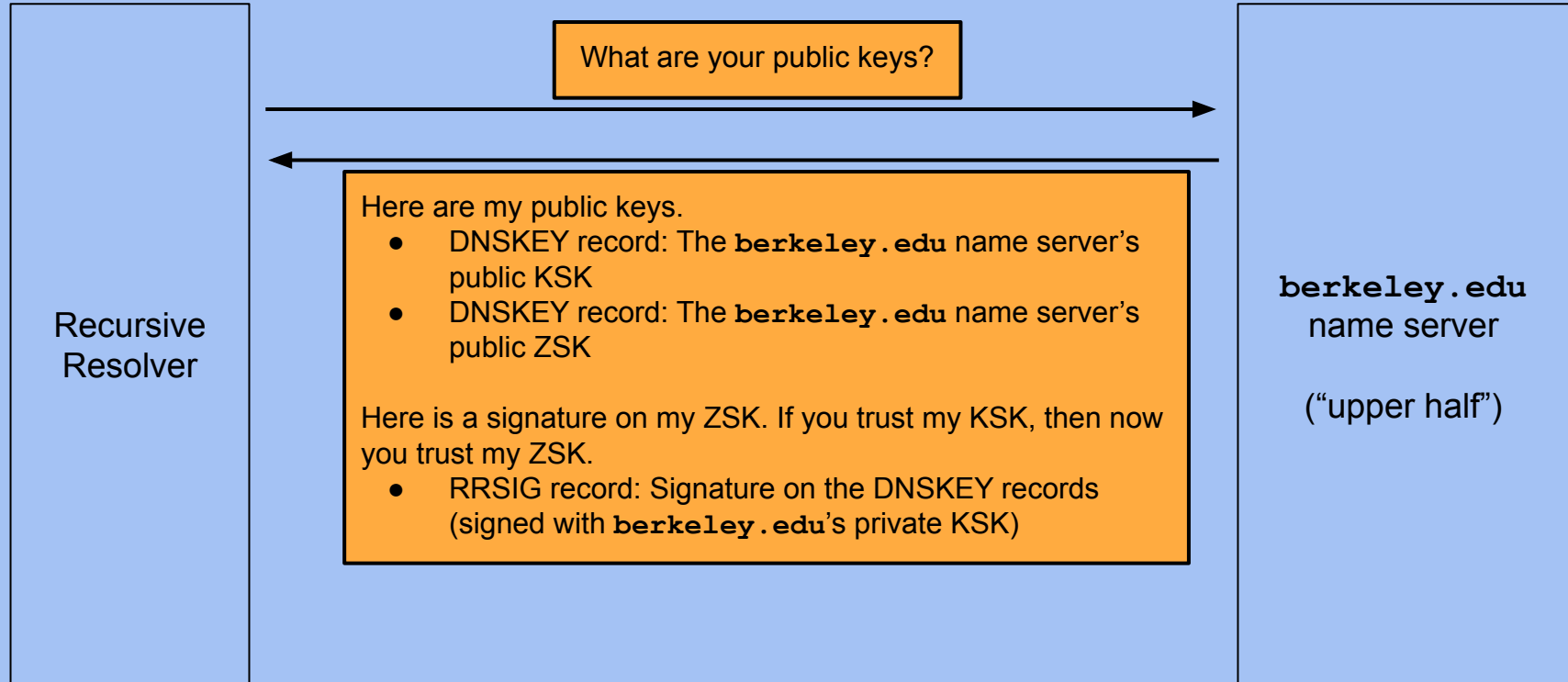
# Steps of a DNSSEC Lookup (Attempt #3)



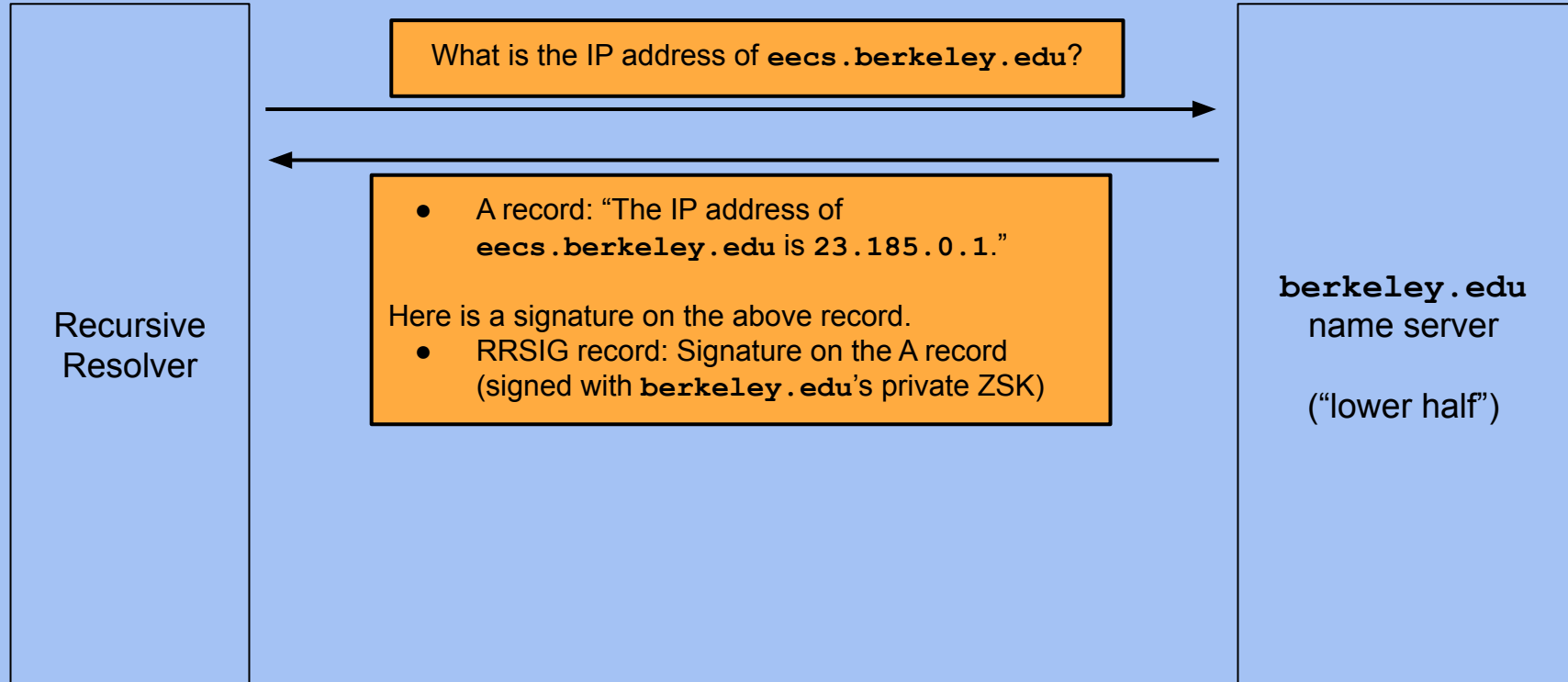
# Steps of a DNSSEC Lookup (Attempt #3)



# Steps of a DNSSEC Lookup (Attempt #3)



# Steps of a DNSSEC Lookup (Attempt #3)



# DNSSEC Lookup Walkthrough

CSE 405 - Computer Security - ART

```
$ dig +norecurse +dnssec DNSKEY . @198.41.0.4
```



You can try this at home! Use the **dig** utility in your terminal, and remember to set the **+norecurse** flag so you can traverse the name server hierarchy yourself and the **+dnssec** flag so that you receive DNSSEC responses.

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec DNSKEY . @198.41.0.4
```

The first step is to query the root name server for its public keys.

*The chain of trust*

| Name | Type                |
|------|---------------------|
| .    | <b>DNSKEY (KSK)</b> |
|      |                     |
|      |                     |
|      |                     |
|      |                     |
|      |                     |
|      |                     |
|      |                     |
|      |                     |

# DNSSEC Lookup Walkthrough

CSE 405 - Computer Security - ART

```
$ dig +norecurse +dnssec DNSKEY . @198.41.0.4

;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7149
;; flags: qr aa; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1472
;; QUESTION SECTION:
; .                IN      DNSKEY

;; ANSWER SECTION:
.      172800      IN      DNSKEY      256 {ZSK of root}
.      172800      IN      DNSKEY      257 {KSK of root}
.      172800      IN      RRSIG       DNSKEY {signature on DNSKEY records}
...
```

| The chain of trust |              |
|--------------------|--------------|
| Name               | Type         |
| .                  | DNSKEY (KSK) |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |

The header says there's 1 record in the additional section, but the additional section is empty! What happened?

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec DNSKEY . @198.41.0.4

;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7149
;; flags: qr aa; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1472
;; QUESTION SECTION:
.                IN      DNSKEY

;; ANSWER SECTION:
.      172800      IN      DNSKEY      256 {ZSK of root}
.      172800      IN      DNSKEY      257 {KSK of root}
.      172800      IN      RRSIG       DNSKEY {signature on DNSKEY records}
...
```

| The chain of trust |              |
|--------------------|--------------|
| Name               | Type         |
| .                  | DNSKEY (KSK) |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |

The additional record is actually the OPT pseudosection, which **dig** lists separately for us.

Note the **do** flag, which indicates that DNSSEC is supported.

# DNSSEC Lookup Walkthrough

CSE 405 - Computer Security - ART

```
$ dig +norecurse +dnssec DNSKEY . @198.41.0.4

;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7149
;; flags: qr aa; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1472
;; QUESTION SECTION:
;.                IN      DNSKEY

;; ANSWER SECTION:
.      172800      IN      DNSKEY    256 {ZSK of root}
.      172800      IN      DNSKEY    257 {KSK of root}
.      172800      IN      RRSIG     DNSKEY {signature on DNSKEY records}
...
```

| The chain of trust |              |
|--------------------|--------------|
| Name               | Type         |
| .                  | DNSKEY (KSK) |
| .                  | DNSKEY (ZSK) |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |
|                    |              |

The root's KSK signs the root's ZSK. If you trust the root's KSK (trust anchor), now you trust the root's ZSK.

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec eecs.berkeley.edu @198.41.0.4
```

*The chain of trust*

| Name | Type                |
|------|---------------------|
| .    | <b>DNSKEY (KSK)</b> |
| .    | DNSKEY (ZSK)        |
|      |                     |
|      |                     |
|      |                     |
|      |                     |
|      |                     |
|      |                     |
|      |                     |

Next, we ask the root name server about the IP address of **eeecs.berkeley.edu**.

# DNSSEC Lookup Walkthrough

CSE 405 - Computer Security - ART

```
$ dig +norecurse +dnssec eecs.b  
;; Got answer:  
;; ->>HEADER<<- opcode: QUERY,  
;; flags: qr; QUERY: 1, ANSWER:  
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags: do;  
;; QUESTION SECTION:  
;eecs.berkeley.edu.
```

```
;; AUTHORITY SECTION:
```

```
edu.          172800   IN      NS      a.edu-servers.net.  
edu.          172800   IN      NS      b.edu-servers.net.  
edu.          172800   IN      NS      c.edu-servers.net.  
...
```

```
edu.          86400    IN      DS      {hash of .edu's KSK}  
edu.          86400    IN      RRSIG   DS {signature on DS record}
```

```
;; ADDITIONAL SECTION:
```

```
a.edu-servers.net. 172800   IN      A      192.5.6.30  
b.edu-servers.net. 172800   IN      A      192.33.14.30  
c.edu-servers.net. 172800   IN      A      192.26.92.30  
...
```

The records are all the same as ordinary DNS, except for these two extra records endorsing the .edu name server's public KSK.

If you trust the root's ZSK, now you trust the .edu name server's KSK.

*The chain of trust*

| Name | Type         |
|------|--------------|
| .    | DNSKEY (KSK) |
| .    | DNSKEY (ZSK) |
| edu. | DS           |
|      |              |
|      |              |
|      |              |
|      |              |
|      |              |

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec DNSKEY edu. @192.5.6.30
```

Next, we query the **.edu** name server for its public keys.

*The chain of trust*

| Name        | Type                |
|-------------|---------------------|
| .           | <b>DNSKEY (KSK)</b> |
| .           | DNSKEY (ZSK)        |
| <b>edu.</b> | DS                  |
|             |                     |
|             |                     |
|             |                     |
|             |                     |
|             |                     |
|             |                     |

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec DNSKEY edu. @192.5.6.30

;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 9776
;; flags: qr aa; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 4096
;; QUESTION SECTION:
;edu.          IN      DNSKEY

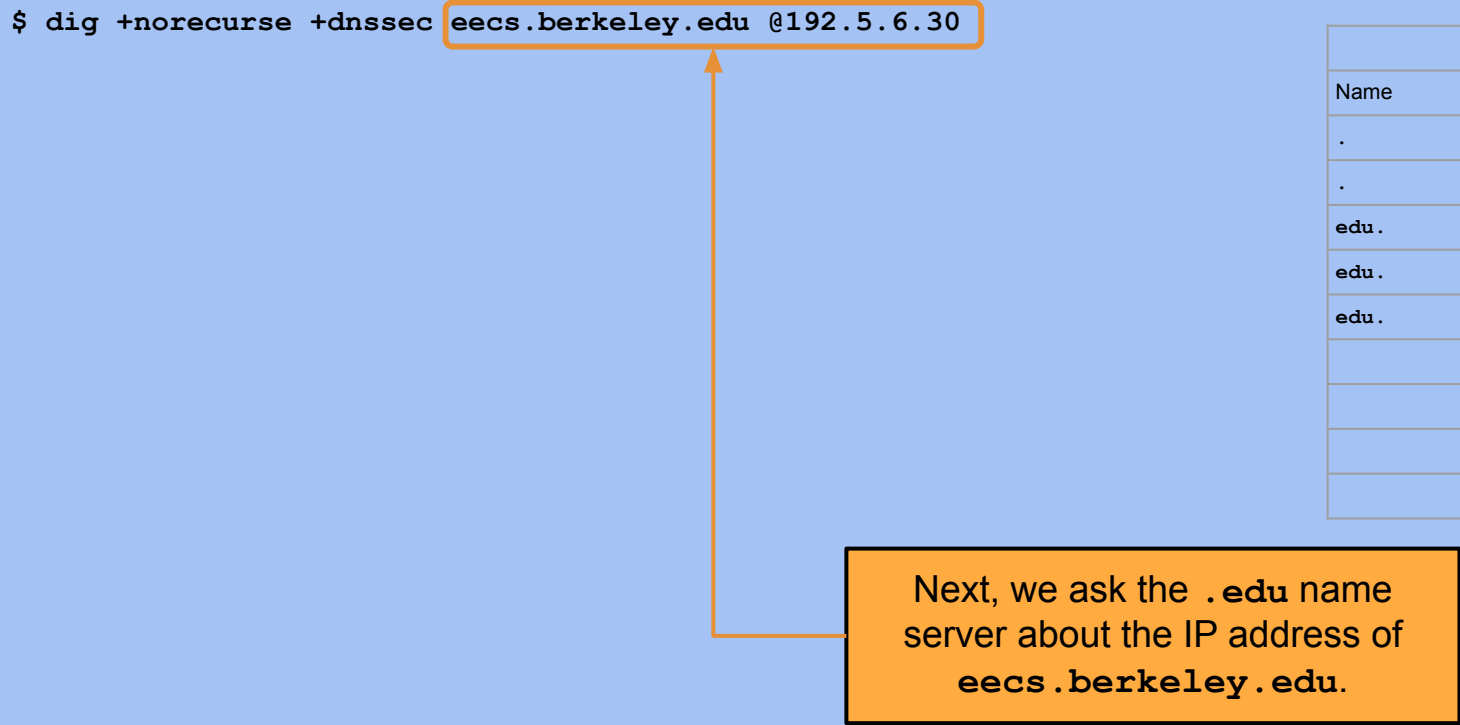
;; ANSWER SECTION:
edu.  86400  IN      DNSKEY  256 {ZSK of .edu}
edu.  86400  IN      DNSKEY  257 {KSK of .edu}
edu.  86400  IN      RRSIG   DNSKEY {signature on DNSKEY records}
...
```

| The chain of trust |              |
|--------------------|--------------|
| Name               | Type         |
| .                  | DNSKEY (KSK) |
| .                  | DNSKEY (ZSK) |
| edu.               | DS           |
| edu.               | DNSKEY (KSK) |
| edu.               | DNSKEY (ZSK) |
|                    |              |
|                    |              |
|                    |              |
|                    |              |

The .edu name server's KSK signs the .edu name server's ZSK. If you trust .edu's KSK, now you trust .edu's ZSK.

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec eecs.berkeley.edu @192.5.6.30
```



The diagram illustrates a DNSSEC lookup process. A terminal command is shown with the domain `eeecs.berkeley.edu` highlighted in an orange box. An orange arrow points from this box down to a callout box. The callout box contains text explaining the next step: asking the `.edu` name server about the IP address of `eeecs.berkeley.edu`.

Next, we ask the `.edu` name server about the IP address of `eeecs.berkeley.edu`.

| The chain of trust |                     |
|--------------------|---------------------|
| Name               | Type                |
| .                  | <b>DNSKEY (KSK)</b> |
| .                  | DNSKEY (ZSK)        |
| edu.               | DS                  |
| edu.               | DNSKEY (KSK)        |
| edu.               | DNSKEY (ZSK)        |
|                    |                     |
|                    |                     |
|                    |                     |
|                    |                     |

# DNSSEC Lookup Walkthrough

CSE 405 - Computer Security - ART

```
$ dig +norecurse +dnssec eecs.berkeley.edu @192.5.6.30
```

```
;; Got answer:
;; ->>HEADER<<- opcode
;; flags: qr; QUERY:
```

```
;; OPT PSEUDOSECTION:
; EDNS: version: 0, f
;; QUESTION SECTION:
;eecs.berkeley.edu.
```

```
;; AUTHORITY SECTION:
```

```
berkeley.edu.      172800  IN  NS      adns1.berkeley.edu.
berkeley.edu.      172800  IN  NS      adns2.berkeley.edu.
berkeley.edu.      172800  IN  NS      adns3.berkeley.edu.
```

```
berkeley.edu.      86400   IN  DS      {hash of berkeley.edu's KSK}
berkeley.edu.      86400   IN  RRSIG   DS {signature on DS record}
```

```
;; ADDITIONAL SECTION:
```

```
adns1.berkeley.edu. 172800  IN  A       128.32.136.3
adns2.berkeley.edu. 172800  IN  A       128.32.136.14
adns3.berkeley.edu. 172800  IN  A       192.107.102.142
```

```
...
```

Again, the records are all the same as ordinary DNS, except for these two extra records endorsing the **berkeley.edu** name server's public KSK.

If you trust the **.edu** name server's ZSK, now you trust the **berkeley.edu** name server's KSK.

*The chain of trust*

| Name          | Type         |
|---------------|--------------|
| .             | DNSKEY (KSK) |
| .             | DNSKEY (ZSK) |
| edu.          | DS           |
| edu.          | DNSKEY (KSK) |
| edu.          | DNSKEY (ZSK) |
| berkeley.edu. | DS           |
|               |              |
|               |              |
|               |              |

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec DNSKEY berkeley.edu @128.32.136.3
```



*The chain of trust*

| Name          | Type                |
|---------------|---------------------|
| .             | <b>DNSKEY (KSK)</b> |
| .             | DNSKEY (ZSK)        |
| edu.          | DS                  |
| edu.          | DNSKEY (KSK)        |
| edu.          | DNSKEY (ZSK)        |
| berkeley.edu. | DS                  |
|               |                     |
|               |                     |
|               |                     |

Next, we query the **berkeley.edu** name server for its public keys.

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec DNSKEY berkeley.edu @128.32.136.3

;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4169
;; flags: qr aa; QUERY: 1, ANSWER: 5, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1220
;; QUESTION SECTION:
;berkeley.edu.      IN   DNSKEY

;; ANSWER SECTION:
berkeley.edu.  172800 IN   DNSKEY  256 {ZSK of berkeley.edu}
berkeley.edu.  172800 IN   DNSKEY  257 {KSK of berkeley.edu}
berkeley.edu.  172800 IN   RRSIG   DNSKEY {signature on DNSKEY records}
...
```

| The chain of trust |              |
|--------------------|--------------|
| Name               | Type         |
| .                  | DNSKEY (KSK) |
| .                  | DNSKEY (ZSK) |
| edu.               | DS           |
| edu.               | DNSKEY (KSK) |
| edu.               | DNSKEY (ZSK) |
| berkeley.edu.      | DS           |
| berkeley.edu.      | DNSKEY (KSK) |
| berkeley.edu.      | DNSKEY (ZSK) |

The **berkeley.edu** name server's KSK signs the **berkeley.edu** name server's ZSK. If you trust **berkeley.edu**'s KSK, now you trust **berkeley.edu**'s ZSK.

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec eecs.berkeley.edu @128.32.136.3
```

| The chain of trust |                     |
|--------------------|---------------------|
| Name               | Type                |
| .                  | <b>DNSKEY (KSK)</b> |
| .                  | DNSKEY (ZSK)        |
| edu.               | DS                  |
| edu.               | DNSKEY (KSK)        |
| edu.               | DNSKEY (ZSK)        |
| berkeley.edu.      | DS                  |
| berkeley.edu.      | DNSKEY (KSK)        |
| berkeley.edu.      | DNSKEY (ZSK)        |
|                    |                     |

Finally, we ask the **berkeley.edu** name server about the IP address of **eeecs.berkeley.edu**.

# DNSSEC Lookup Walkthrough

```
$ dig +norecurse +dnssec eecs.berkeley.edu @128.32.136.3

;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 21205
;; flags: qr aa; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1220
;; QUESTION SECTION:
;eecs.berkeley.edu.          IN      A

;; ANSWER SECTION:
eecs.berkeley.edu.  86400   IN      A          23.185.0.1
eecs.berkeley.edu.  86400   IN      RRSIG   A {signature on A record}
```

| The chain of trust |              |
|--------------------|--------------|
| Name               | Type         |
| .                  | DNSKEY (KSK) |
| .                  | DNSKEY (ZSK) |
| edu.               | DS           |
| edu.               | DNSKEY (KSK) |
| edu.               | DNSKEY (ZSK) |
| berkeley.edu.      | DS           |
| berkeley.edu.      | DNSKEY (KSK) |
| berkeley.edu.      | DNSKEY (ZSK) |
| eecs.berkeley.edu. | A            |

Here's the final answer record, signed by **berkeley.edu**'s public ZSK. If you trust **berkeley.edu**'s ZSK, then now you trust the final answer.

# NSEC: Signing Non-Existent Domains

# Nonexistent Domains

- The DNSSEC structure works great for domains which exist
  - We have signatures over records stating that they exist
- What if the user queries for a domain that **doesn't** exist?
  - Option #1: Don't authenticate nonexistent domain (NXDOMAIN) responses
    - Issue: If NXDOMAIN responses don't have to be signed, the attacker can still spoof NXDOMAIN responses and cause denial-of-service (DoS)
  - Option #2: Keep the private key in the name server itself, so it signs NXDOMAIN responses
    - Issue: Name servers have access to the private key, which is an issue if they are malicious or hacked
    - Issue: Signing in real time is slow
  - We need a way that can prove that a domain doesn't exist ahead of time

# NSEC: Authenticated Denial of Existence

- Prove nonexistence of a record type
  - Sign a record stating that no record of a given type exists
  - Useful for proving that a domain doesn't support DNSSEC ("No DS records exist")
- Prove nonexistence of a domain
  - Provide two adjacent domains alphabetically, so that you know that no domain in the middle exists
  - Example: If I query for `nonexistent.google.com`, I can receive a signed NSEC response saying "No domains exist between `maps.google.com` and `one.google.com`."
  - We can sign all pairs of adjacent records ahead of time and keep them as NSEC records, along with their RRSIGs

---

`maps`

`one`

`web`

# Issues with NSEC

- **Domain enumeration:** It is easy for an attacker to find every single subdomain of a domain
  - Start by querying `a.google.com`
  - Receive an NSEC record stating that “No domains exist between `web.google.com` and `apps.google.com`”
    - Now we have learned two domain names!
  - Repeat: query `appsa.google.com` (alphabetically immediately after `apps.google.com`)
  - Receive an NSEC record stating that “No domains exist between `apps.google.com` and `map.google.com`”
  - Repeat until you loop back around to the beginning

---

`apps`

`map`

`one`

`web`

# NSEC3: Hashed Authenticated Denial of Existence

- Idea: Instead of storing pairs of adjacent domain names, store pairs of adjacent hashes
  - Example: If I query for `nonexistent.google.com`, which hashes to `d48678...`, I receive a signed NSEC3 saying “There exist no domains which hash to values between `c612f3...` and `d810de...`”

---

`c612f3`

`d810de`

# Issues with NSEC3

- Domain enumeration is still possible since most people choose short domain names
  - Possible to brute-force through all reasonable domain names!
  - Only prevents attackers from learning long, random domain names, which would make brute-force difficult
- The only real way to prevent enumeration is online signature generation with the private key
  - Coming down the pipeline: NSEC5

# NSEC3 (“White Lies”): Minimal proof on-the-fly

- Idea: Given a nonexistent domain **name**, generate a signature on-the-fly that states that no record exists between  $H(\text{name}) - 1$  and  $H(\text{name}) + 1$ 
  - Example: If I query for **evanbot.google.com**, which hashes to ...mqdgm6, I receive a signed NSEC3 saying “There exist no domains which hash to values between ...mqdgm5 and ...mqdgm7”
- Solves our domain enumeration problem!

---

mqdgm5

mqdgm7

# Issues with NSEC3 (“White Lies”)

- NSEC3 (“white lies”) requires online signature generation
  - Cannot precompute every possible name/hash offline and don’t have the storage capacity to do so for every single name server!
  - This is a lot of overhead, so it isn’t really broadly used!
- Doesn’t abide by our design requirement (we don’t want to keep the private key in the name server itself!)

# DNSSEC in Practice

# Offline Signature Generation

- Offline signatures: The application that computes signatures is separate from the application that serves the signatures
- Benefit: Efficiency
  - Records are signed ahead of time, and the signature is stored and served on request
  - Generating a signature each time a user requests it is slow (and can lead to DoS attacks)
- Benefit: Security
  - An attacker must compromise the signature generation system (e.g. steal the private signing key) to perform an attack
  - If the signature generation system is separate from the name server, compromising the name server is not enough!
  - Redundancy: One secure signature generation system, and many *mirrored* name servers providing the same records and signatures

# Efficiency: Parallelization

- Requests can be made in parallel to improve performance
  - Example: Request DNSKEY records from every name server in parallel
- Signatures can be validated in parallel
  - Example: Validate the parent's DS record while waiting for the child's DNSKEY record

# Implementation Errors

- Implementation errors from the name servers
  - Example: A name server claims to support DNSSEC, even though it doesn't
  - Example: Changing your key but presenting old signatures signed with an old key
  - Example: Present expired signatures
- Implementation errors from the resolvers
  - The resolver can't access DNSSEC records
  - The resolver can't process DNSSEC records correctly

# Implementation Errors: Examples

- The launch of HBO Go (a streaming service) was broken for Comcast users and users using Google Public DNS
  - The DNS servers reported that they supported DNSSEC when they didn't
- Google Public DNS and Comcast provide recursive resolvers
  - When a name server messes up, Comcast and Google are often blamed
  - Fortunately, this is getting less common
- An educational network had several mirrors of a name server
  - 3 mirrors supported DNSSEC. All other mirrors didn't support DNSSEC
- Wi-Fi hotspots (e.g. at Starbucks) often proxy DNS
  - Proxy: Receive a DNS request and replace it with its own DNS request
  - The proxy often doesn't support DNSSEC

# Implementation Error: Incomplete Validation

- Most DNSSEC implementations only validate records at the recursive resolver, not the client (stub resolver)
- If the client doesn't validate records, the recursive resolver can poison the cache!
  - Recall: The recursive resolver is the biggest threat in DNS
- If the client doesn't validate records, network attackers can still poison the cache!
  - Example: An on-path attacker between the recursive resolver and the client
- Result: If the client doesn't validate records, DNSSEC provides very little practical security

# DNSSEC: Summary

- DNSSEC: An extension of the DNS protocol that ensures integrity on the results
  - Provides object security (unlike DNS over TLS, which would provide channel security)
  - Uses signatures to cryptographically verify records
  - Uses a hierarchical public key infrastructure to delegate trust from the trust anchor (root)
- DNSSEC Implementation
  - Each name server replies with its public key (**DNSKEY** type)
  - When delegating trust, each name server signs the public key of the next name server (**DS** and **RRSIG** types)
  - When providing a final answer, the name server signs the final answer (**RRSIG** type)
  - Zones are split into key-signing keys and zone-signing keys
  - NSEC signs a message saying no domains exist alphabetically between two records

# DNSSEC: Summary

- DNSSEC: An extension of the DNS protocol that ensures integrity on the results
  - Provides object security (unlike DNS over TLS, which would provide channel security)
  - Uses signatures to cryptographically verify records
  - Uses a hierarchical public key infrastructure to delegate trust from the trust anchor (root)